Assignment 08

JSON

1. **Question :** Parse the JSON using JSONObject and print:
    a. Each date
    b. OrderId and CustomerName under each date
2. Create required POJO classes and deserialize the JSON using Gson.
3. For each order:
    a. Calculate the total order amount
    b. Display the payment status
4. Print the following:
    a. All failed orderIds
    b. Total revenue from successful orders
    c. OrderId with the highest order value
5. Date-Based Operations
    a. Calculate total revenue per date
    b. Identify the date with the highest revenue
    c. Print all orderIds for a given input date

Custom JSON Made :


```
{
  "ordersByDate": [
    {
      "date": "2026-01-18",
      "orders": [
        {
          "orderId": "ORD3001",
          "customerName": "Amit",
          "paymentStatus": "SUCCESS",
          "deliveryStatus": "DELIVERED",
          "products": [
            { "name": "Mobile", "quantity": 1, "price": 15000 },
            { "name": "Phone Case", "quantity": 2, "price": 500 }
          ],
          "discount": 0
        },
        {
          "orderId": "ORD3002",
          "customerName": "Neha",
          "paymentStatus": "FAILED",
          "products": [
            { "name": "Laptop", "quantity": 1, "price": 62000 }
          ],
          "discount": 0
        },
```


has extra parameters : discount and delivery status

Output

```
"C:\Program Files\Java\jdk-11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3.1.1\lib\idea_rt.jar=60
Date: 2026-01-18
OrderId: ORD3001, Customer: Amit, Delivery: DELIVERED
No Discount Offered!
OrderId: ORD3002, Customer: Neha, Delivery: N/A
No Discount Offered!
OrderId: ORD3010, Customer: Sohail, Delivery: SHIPPED
Discount Offered: 200.0
Date: 2026-01-19
OrderId: ORD3003, Customer: Ravi, Delivery: DELIVERED
No Discount Offered!
OrderId: ORD3004, Customer: Pooja, Delivery: PENDING
Discount Offered: 100.0
OrderId: ORD3011, Customer: Manish, Delivery: N/A
No Discount Offered!
Date: 2026-01-20
OrderId: ORD3005, Customer: Karan, Delivery: N/A
No Discount Offered!
OrderId: ORD3006, Customer: Sneha, Delivery: DELIVERED
Discount Offered: 50.0
```

```
No Discount Offered!
OrderId: ORD3030, Customer: Vikas, Delivery: PENDING
Discount Offered: 150.0
Using Gson
Date: 2026-01-18
   OrderId: ORD3001   CustomerName: Amit   DeliveryStatus: DELIVERED   PaymentStatus: SUCCESS   TotalAmount: 16000.0
   OrderId: ORD3002   CustomerName: Neha   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 62000.0
   OrderId: ORD3010   CustomerName: Sohail   DeliveryStatus: SHIPPED   PaymentStatus: SUCCESS   TotalAmount: 3200.0
Date: 2026-01-19
   OrderId: ORD3003   CustomerName: Ravi   DeliveryStatus: DELIVERED   PaymentStatus: SUCCESS   TotalAmount: 23200.0
   OrderId: ORD3004   CustomerName: Pooja   DeliveryStatus: PENDING   PaymentStatus: SUCCESS   TotalAmount: 9400.0
   OrderId: ORD3011   CustomerName: Manish   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 3200.0
Date: 2026-01-20
   OrderId: ORD3005   CustomerName: Karan   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 48000.0
   OrderId: ORD3006   CustomerName: Sneha   DeliveryStatus: DELIVERED   PaymentStatus: SUCCESS   TotalAmount: 13950.0
   OrderId: ORD3007   CustomerName: Arjun   DeliveryStatus: SHIPPED   PaymentStatus: SUCCESS   TotalAmount: 10900.0
   OrderId: ORD3008   CustomerName: Meera   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 6400.0
   OrderId: ORD3009   CustomerName: Vikas   DeliveryStatus: DELIVERED   PaymentStatus: SUCCESS   TotalAmount: 8000.0
   OrderId: ORD3012   CustomerName: Isha   DeliveryStatus: PENDING   PaymentStatus: SUCCESS   TotalAmount: 2550.0
Date: 2026-01-21
   OrderId: ORD3013   CustomerName: Rhea   DeliveryStatus: DELIVERED   PaymentStatus: SUCCESS   TotalAmount: 4800.0
   OrderId: ORD3014   CustomerName: Rahul   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 22000.0
```

```
   OrderId: ORD3015   CustomerName: Tina   DeliveryStatus: SHIPPED   PaymentStatus: SUCCESS   TotalAmount: 66200.0
   OrderId: ORD3016   CustomerName: Kabir   DeliveryStatus: PENDING   PaymentStatus: SUCCESS   TotalAmount: 3500.0
   OrderId: ORD3017   CustomerName: Sara   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 12000.0
Date: 2026-01-22
   OrderId: ORD3018   CustomerName: Aditya   DeliveryStatus: DELIVERED   PaymentStatus: SUCCESS   TotalAmount: 28000.0
   OrderId: ORD3019   CustomerName: Neha   DeliveryStatus: SHIPPED   PaymentStatus: SUCCESS   TotalAmount: 9700.0
   OrderId: ORD3020   CustomerName: Vikram   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 25000.0
   OrderId: ORD3021   CustomerName: Isha   DeliveryStatus: PENDING   PaymentStatus: SUCCESS   TotalAmount: 8500.0
Date: 2026-01-23
   OrderId: ORD3022   CustomerName: Rohan   DeliveryStatus: DELIVERED   PaymentStatus: SUCCESS   TotalAmount: 10900.0
   OrderId: ORD3023   CustomerName: Ananya   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 3200.0
   OrderId: ORD3024   CustomerName: Kabir   DeliveryStatus: SHIPPED   PaymentStatus: SUCCESS   TotalAmount: 4950.0
Date: 2026-01-24
   OrderId: ORD3025   CustomerName: Sanya   DeliveryStatus: DELIVERED   PaymentStatus: SUCCESS   TotalAmount: 12200.0
   OrderId: ORD3026   CustomerName: Rajat   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 22000.0
   OrderId: ORD3027   CustomerName: Neha   DeliveryStatus: SHIPPED   PaymentStatus: SUCCESS   TotalAmount: 4600.0
Date: 2026-01-25
   OrderId: ORD3028   CustomerName: Arjun   DeliveryStatus: DELIVERED   PaymentStatus: SUCCESS   TotalAmount: 14500.0
   OrderId: ORD3029   CustomerName: Meera   DeliveryStatus: null   PaymentStatus: FAILED   TotalAmount: 1200.0
   OrderId: ORD3030   CustomerName: Vikas   DeliveryStatus: PENDING   PaymentStatus: SUCCESS   TotalAmount: 9350.0
```

```
  OrderId: ORD3030  CustomerName: Vikas  DeliveryStatus: PENDING  PaymentStatus: SUCCESS  TotalAmount: 9350.
All Failed OrderIds: [ORD3002, ORD3011, ORD3005, ORD3008, ORD3014, ORD3017, ORD3020, ORD3023, ORD3026, ORD3029]
Total Revenue from Successful Orders: 264400.0
OrderId with Highest Order Value: ORD3015 (Amount: 66200.0)

Revenue Per Date:
  Date: 2026-01-18, Revenue: 19200.0
  Date: 2026-01-19, Revenue: 32600.0
  Date: 2026-01-20, Revenue: 35400.0
  Date: 2026-01-21, Revenue: 74500.0
  Date: 2026-01-22, Revenue: 46200.0
  Date: 2026-01-23, Revenue: 15850.0
  Date: 2026-01-24, Revenue: 16800.0
  Date: 2026-01-25, Revenue: 23850.0
Date with Highest Revenue: 2026-01-21 (Revenue: 74500.0)

Enter a date (YYYY-MM-DD) to list all orderIds: 2026-02-29
No orders found for date: 2026-02-29

Process finished with exit code 0
```

Code :

```java
package org.example;
import java.io.FileReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.*;
import com.google.gson.Gson;
import org.json.JSONObject;
import org.json.JSONArray;
import org.json.simple.parser.JSONParser;
public class Main
{
public static String getDeliveryStatus(JSONObject order) {
        return order.has("deliveryStatus") ? order.getString("deliveryStatus") : "N/A";
    }

    public static double getDiscount(JSONObject order) {
        return order.has("discount") ? order.getDouble("discount") : 0.0;
    }

    public static void main(String[] args) throws IOException {
        String json1 = new
String(Files.readAllBytes(Paths.get("src/main/resources/demo.json")));
        JSONObject obj = new JSONObject(json1);
        JSONArray ordersByDate = obj.getJSONArray("ordersByDate");
        for (int i = 0; i < ordersByDate.length(); i++) {
            JSONObject dateObject = ordersByDate.getJSONObject(i);
            String date = dateObject.getString("date");
            System.out.println("Date: " + date);
            JSONArray orders = dateObject.getJSONArray("orders");
```

```
                for (int j = 0; j < orders.length(); j++) {
                    JSONObject order = orders.getJSONObject(j);
                    System.out.println("OrderId: " + order.getString("orderId") +
                            ", Customer: " + order.getString("customerName") +
                            ", Delivery: " + getDeliveryStatus(order));

                    double discount = getDiscount(order);
                    if(discount == 0.0) System.out.println("No Discount Offered!");
                    else System.out.println("Discount Offered: " + discount);
                }

            }

            //GSON
            System.out.println("Using Gson");
            Gson gson = new Gson();
            OrdersByDate ordersByDateObj = gson.fromJson(json1, OrdersByDate.class);
            for (OrdersByDate.OrderDateGroup dateGroup :
ordersByDateObj.getOrdersByDate()) {
                System.out.println("Date: " + dateGroup.getDate());
                for (OrdersByDate.Order order : dateGroup.getOrders()) {
                    System.out.print("   OrderId: " + order.getOrderId());
                    System.out.print("   CustomerName: " + order.getCustomerName());
                    System.out.print("   DeliveryStatus: " + order.getDeliveryStatus());
                    System.out.print("   PaymentStatus: " + order.getPaymentStatus());
                    System.out.print("   TotalAmount: " + order.getTotalAmount());
                    System.out.println();
                }
            }

        List<String> failedOrderIds;
         failedOrderIds = new ArrayList<>();
         double totalRevenueSuccess = 0;
         String highestOrderId = "";
         double highestOrderValue = 0;
         Map<String, Double> revenuePerDate = new TreeMap<>();

        for (OrdersByDate.OrderDateGroup dateGroup : ordersByDateObj.getOrdersByDate()) {
             double dateRevenue = 0;

             for (OrdersByDate.Order order : dateGroup.getOrders()) {
                 double orderTotal = order.getTotalAmount();
                 //failed orders
                 if (!"SUCCESS".equalsIgnoreCase(order.getPaymentStatus())) {
                     failedOrderIds.add(order.getOrderId());
                 } else {
                     totalRevenueSuccess += orderTotal;
                     dateRevenue += orderTotal;
```

```
                }
                //highest order
                if (orderTotal > highestOrderValue) {
                    highestOrderValue = orderTotal;
                    highestOrderId = order.getOrderId();
                }
            }

            revenuePerDate.put(dateGroup.getDate(), dateRevenue);
        }

        String highestRevenueDate = null;
        double maxDateRevenue = 0;
        for (Map.Entry<String, Double> entry : revenuePerDate.entrySet()) {
            if (entry.getValue() > maxDateRevenue) {
                maxDateRevenue = entry.getValue();
                highestRevenueDate = entry.getKey();
            }
        }

        System.out.println("All Failed OrderIds: " + failedOrderIds);
        System.out.println("Total Revenue from Successful Orders: " +
totalRevenueSuccess);
        System.out.println("OrderId with Highest Order Value: " + highestOrderId + "
(Amount: " + highestOrderValue + ")");
        System.out.println("\nRevenue Per Date:");
        for (Map.Entry<String, Double> entry : revenuePerDate.entrySet()) {
            System.out.println("  Date: " + entry.getKey() + ", Revenue: " +
entry.getValue());
        }
        System.out.println("Date with Highest Revenue: " + highestRevenueDate + "
(Revenue: " + maxDateRevenue + ")");

       Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter a date (YYYY-MM-DD) to list all orderIds: ");
        String inputDate = sc.nextLine();

        boolean dateFound = false;
        for (OrdersByDate.OrderDateGroup dateGroup : ordersByDateObj.getOrdersByDate()) {
            if (dateGroup.getDate().equals(inputDate)) {
                dateFound = true;
                System.out.println("OrderIds for " + inputDate + ":");
                for (OrdersByDate.Order order : dateGroup.getOrders()) {
                    System.out.println("  " + order.getOrderId());
                }
                break;
            }
        }
```

```java
        if (!dateFound) {
            System.out.println("No orders found for date: " + inputDate);
        }
        sc.close();
        }
    }

package org.example;
import java.util.List;

public class OrdersByDate {
    private List<OrderDateGroup> ordersByDate;
    public List<OrderDateGroup> getOrdersByDate() {
        return ordersByDate;
    }

    public static class OrderDateGroup {
        private String date;
        private List<Order> orders;
        public String getDate() { return date; }
        public List<Order> getOrders() { return orders; }
    }

    public static class Order {
        private String orderId;
        private String customerName;
        private String paymentStatus;
        private String deliveryStatus;
        private Double discount;
        private List<Product> products;
        public String getOrderId() { return orderId; }
        public String getCustomerName() { return customerName; }
        public String getPaymentStatus() { return paymentStatus; }
        public String getDeliveryStatus() { return deliveryStatus; }
        public Double getDiscount() { return discount != null ? discount : 0; }
        public List<Product> getProducts() { return products; }
        public double getTotalAmount() {
            double total = 0;
            for (Product p : products) {
                total += p.getPrice() * p.getQuantity();
            }
            return total - getDiscount();
        }
    }
    public static class Product {
        private String name;
        private int quantity;
        private double price;
        public String getName() { return name; }
```

```
        public int getQuantity() { return quantity; }
        public double getPrice() { return price; }
    }
}
```

```
package org.example;
import java.util.List;

public class OrdersByDate {
    private List<OrderDateGroup> ordersByDate;
    public List<OrderDateGroup> getOrdersByDate() {
        return ordersByDate;
    }

    public static class OrderDateGroup {
        private String date;
        private List<Order> orders;
        public String getDate() { return date; }
        public List<Order> getOrders() { return orders; }
    }

    public static class Order {
        private String orderId;
        private String customerName;
        private String paymentStatus;
        private String deliveryStatus;
        private Double discount;
        private List<Product> products;
        public String getOrderId() { return orderId; }
        public String getCustomerName() { return customerName; }
        public String getPaymentStatus() { return paymentStatus; }
        public String getDeliveryStatus() { return deliveryStatus; }
        public Double getDiscount() { return discount != null ? discount : 0; }
        public List<Product> getProducts() { return products; }
        public double getTotalAmount() {
            double total = 0;
            for (Product p : products) {
                total += p.getPrice() * p.getQuantity();
            }
            return total - getDiscount();
        }
    }
    public static class Product {        private String name;
        private int quantity;
        private double price;
        public String getName() { return name; }
```

```
        public int getQuantity() { return quantity; }
        public double getPrice() { return price; }
    }
}
```

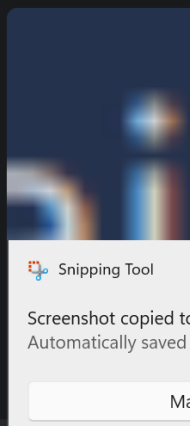
Code Snippet

Main class

```
Main.java × | OrdersByDate.java | sample.json | demo.json | pom.xml (JsonAssignment)
package org.example;
import java.io.FileReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.*;
import com.google.gson.Gson;
import org.json.JSONObject;
import org.json.JSONArray;
import org.json.simple.parser.JSONParser;
public class Main {
    public static String getDeliveryStatus(JSONObject order) {  1 usage
        return order.has( key: "deliveryStatus") ? order.getString( key: "deliveryStatus") : "N/A";
    }

    public static double getDiscount(JSONObject order) {  1 usage
        return order.has( key: "discount") ? order.getDouble( key: "discount") : 0.0;
    }

    public static void main(String[] args) throws IOException {
        String json1 = new String(Files.readAllBytes(Paths.get( first: "src/main/resources/demo.json")));
        JSONObject obj = new JSONObject(json1);
        JSONArray ordersByDate = obj.getJSONArray( key: "ordersByDate");
        for (int i = 0; i < ordersByDate.length(); i++) {
            JSONObject dateObject = ordersByDate.getJSONObject(i);
```

```java
public static void main(String[] args) throws IOException {
    for (int i = 0; i < ordersByDate.length(); i++) {
        JSONObject dateObject = ordersByDate.getJSONObject(i);
        String date = dateObject.getString( key: "date");
        System.out.println("Date: " + date);
        JSONArray orders = dateObject.getJSONArray( key: "orders");
            for (int j = 0; j < orders.length(); j++) {
                JSONObject order = orders.getJSONObject(j);
                System.out.println("OrderId: " + order.getString( key: "orderId") +
                        ", Customer: " + order.getString( key: "customerName") +
                        ", Delivery: " + getDeliveryStatus(order));

                double discount = getDiscount(order);
                if(discount == 0.0) System.out.println("No Discount Offered!");
                else System.out.println("Discount Offered: " + discount);
            }}
        //GSON
        System.out.println("Using Gson");
        Gson gson = new Gson();
        OrdersByDate ordersByDateObj = gson.fromJson(json1, OrdersByDate.class);
        for (OrdersByDate.OrderDateGroup dateGroup : ordersByDateObj.getOrdersByDate()) {
            System.out.println("Date: " + dateGroup.getDate());
            for (OrdersByDate.Order order : dateGroup.getOrders()) {
                System.out.print("   OrderId: " + order.getOrderId());
```

```java
                System.out.print("   DeliveryStatus: " + order.getDeliveryStatus());
                System.out.print("   PaymentStatus: " + order.getPaymentStatus());
                System.out.print("   TotalAmount: " + order.getTotalAmount());
                System.out.println();
            }
        }

    List<String> failedOrderIds;
    failedOrderIds = new ArrayList<>();
    double totalRevenueSuccess = 0;
    String highestOrderId = "";
    double highestOrderValue = 0;
    Map<String, Double> revenuePerDate = new TreeMap<>();

    for (OrdersByDate.OrderDateGroup dateGroup : ordersByDateObj.getOrdersByDate()) {
        double dateRevenue = 0;

        for (OrdersByDate.Order order : dateGroup.getOrders()) {
            double orderTotal = order.getTotalAmount();
            //failed orders
            if (!"SUCCESS".equalsIgnoreCase(order.getPaymentStatus())) {
                failedOrderIds.add(order.getOrderId());
            } else {
```



```java
        public static void main(String[] args) throws IOException {
            System.out.println("Date with Highest Revenue: " + highestRevenueDate + " (Revenue: " + maxDateRevenue + ")");

            Scanner sc = new Scanner(System.in);
            System.out.print("\nEnter a date (YYYY-MM-DD) to list all orderIds: ");
            String inputDate = sc.nextLine();

            boolean dateFound = false;
            for (OrdersByDate.OrderDateGroup dateGroup : ordersByDateObj.getOrdersByDate()) {
                if (dateGroup.getDate().equals(inputDate)) {
                    dateFound = true;
                    System.out.println("OrderIds for " + inputDate + ":");
                    for (OrdersByDate.Order order : dateGroup.getOrders()) {
                        System.out.println("  " + order.getOrderId());
                    }
                    break;
                }
            }
            if (!dateFound) {
                System.out.println("No orders found for date: " + inputDate);
            }
            sc.close();
            }
        }
```